

Mini Golf: A Line-by-Line Guide to the Code

`types.h`, `input`, `level`, `putter` and the menu in `main.c`

Golf-Game project

September 25, 2026

Contents

1	Introduction	3
2	<code>types.h</code>	3
2.1	Include guard and constants	3
2.2	<code>SurfaceType</code>	4
2.3	<code>BallState</code> and <code>GameStateId</code>	4
2.4	<code>Ball</code>	5
2.5	<code>Wall</code> and <code>Zone</code>	5
2.6	<code>Hole</code>	6
2.7	<code>Game</code>	7
3	<code>input.h</code> and <code>input.c</code>	7
3.1	<code>input.h</code>	7
3.2	Constants, <code>DirFromAngle</code> and <code>InputInit</code>	8
3.3	<code>InputPoll</code>	9
4	<code>level.h</code> and <code>level.c</code>	11
4.1	<code>level.h</code>	11
4.2	The level file format	12
4.3	<code>AddZone</code> , <code>Validate_hole</code> , <code>AddWall</code>	12
4.4	<code>Parseline</code>	13
4.5	<code>LoadHoleFromFile</code>	15
4.6	<code>SaveHoleToFile</code>	15
4.7	The course	16
4.8	Scoring	16
4.9	The editor	17
5	<code>putter.h</code> and <code>putter_simple.c</code>	20
5.1	<code>putter.h</code>	20
5.2	<code>putter_simple.c</code>	21
6	The menu in <code>main.c</code>	23
6.1	<code>ScreenState</code>	23
6.2	<code>GameApp</code>	24
6.3	<code>DrawFullscreenTexture</code>	24
6.4	<code>DrawMenuButton</code>	24
6.5	<code>StartHole</code>	25
6.6	Music functions	25
6.7	<code>GameInit</code>	26

6.8	<code>DrawMainMenu</code>	27
6.9	Tutorial and About screens	28
6.10	The main loop: choosing the screen	28
6.11	Cleaning up	29

1 Introduction

This document walks through every line of five parts of the game:

- `types.h` – the data types the whole game shares;
- `input.h` / `input.c` – reading the mouse;
- `level.h` / `level.c` – loading maps, the course, scoring and the editor;
- `putter.h` / `putter_simple.c` – turning a mouse click into a shot;
- the menu part of `main.c`.

All code listings are taken directly from the source files, and the line numbers on the left are the real line numbers in each file. So “line 36 of `input.c`” in the text is line 36 in your editor too.

The game uses the **raylib** library. A few raylib types appear everywhere:

raylib type	meaning
<code>Vector2</code>	a 2D point or direction: <code>{float x, float y}</code>
<code>Rectangle</code>	<code>{float x, float y, float width, float height}</code> , (x, y) is the top-left corner
<code>Camera2D</code>	where the camera looks, its zoom and rotation
<code>Texture2D</code>	an image loaded onto the graphics card
<code>Music</code>	a streamed audio file (mp3, ogg, ...)

The screen’s y axis points *down*, so “up” on the screen is negative y .

2 types.h

`types.h` has no functions. It only defines constants, enums and structs that every other file uses, so all modules agree on what a ball, a wall or a hole looks like.

2.1 Include guard and constants

```
1 #ifndef TYPES_H //In order to prevent redundancy. Such as physics.h also includes this "types.  
    ↪ h", and i am including both types.h and physics.h inside ball.h, so all of the data  
    ↪ will be defined twice.  
2 #define TYPES_H // If "types.h" is not defined then at first then define types.h, and if it  
    ↪ is already defined, then don't open this file  
3  
4 #include "raylib.h"  
5  
6 //Define  
7 #define MAX_WALLS 128  
8 #define MAX_ZONES 32  
9 #define MAX_HOLES 18  
10 #define MAX_OBSTACLES 32
```

Lines 1–2, 109. This is an *include guard*. The first time the compiler reads the file, `TYPES_H` is not defined, so it defines it and reads the rest. If another header includes `types.h` again (for example `ball.h` and `level.h` both include it), `TYPES_H` is already defined and the whole file is skipped up to the `#endif` on line 109. Without this, every struct would be defined twice and the program would not compile.

Line 4. Includes raylib so that `Vector2`, `Rectangle` and `Camera2D` are known.

Lines 7–10. Maximum sizes. A hole can have at most 128 walls and 32 zones, and a course at most 18 holes. These are used as array sizes, so no dynamic memory (`malloc`) is needed. `MAX_OBSTACLES` is not used anywhere yet.

2.2 SurfaceType

```
12 //SURFACE TYPES
13 typedef enum
14 {
15     SURF_GREEN,
16     SURF_FAIRWAY,
17     SURF_SAND,
18     SURF_ICE,
19     SURF_MUD,
20     SURF_WATER,
21     SURF_BOOST,
22     SURF_COUNT, /* it will automatically update the surface numbers that i have created
    ↪ .....(rafin)
23                 so next time we can just add a new surface, without changing the count*/
24 }SurfaceType;
```

An `enum` gives names to integers. The first name is 0 and each next one is one larger:

value	name	friction	behaviour (defined in <code>physics.c</code>)
0	<code>SURF_GREEN</code>	400	the default ground where there is no zone
1	<code>SURF_FAIRWAY</code>	300	a little faster than green
2	<code>SURF_SAND</code>	1400	slows the ball a lot
3	<code>SURF_ICE</code>	90	very slippery
4	<code>SURF_MUD</code>	2600	strongest brake
5	<code>SURF_WATER</code>	–	hazard: the ball is dropped and gets +1 stroke
6	<code>SURF_BOOST</code>	–600	negative friction, so it speeds the ball up

Line 22. `SURF_COUNT` is a trick: because it comes last, its value is 7, which is exactly the number of real surfaces. Arrays like `kfriction[SURF_COUNT]` and loops like `(zone_type + 1) % SURF_COUNT` use it. If you add a new surface before it, the count updates itself.

The numbers in the table are what you write after `zone` in a level file.

2.3 BallState and GameStateId

```
27 //Ballstate-condition of the ball
28 typedef enum
29 {
30     BALL_AIM,
31     BALL_CHARGE,
32     BALL_ROLL,
33     BALL_SUNK,
34     BALL_OOB,
35 }BallState;
36 //OOB - Out of bounds
37
38 //Defining current state of the game
39 typedef enum
40 {
41     GS_MENU, // shows menu or options
42     GS_PLAYING, //player is at playing mode
```

```

43     GS_HOLE_DONE, //shows hole completed informations
44     GS_SCOREBOARD, //shows scores or results
45     GS_PAUSED // stops gameplay while showing pause menu
46 }GameStateId;

```

Lines 28–35, BallState. What the ball is doing right now:

- BALL_AIM – resting, the player may hit it;
- BALL_CHARGE – declared, but the putter does not use it (the putter has its own phase);
- BALL_ROLL – moving, physics runs every frame;
- BALL_SUNK – in the cup, the hole is finished;
- BALL_OOB – “out of bounds”: in water or outside the field; it will be dropped.

Lines 39–46, GameStateId. The state of a round. `main.c` uses `GS_PLAYING` (hitting the ball), `GS_HOLE_DONE` (the short pause showing “BIRDIE” etc.) and `GS_SCOREBOARD` (after the last hole). `GS_MENU` and `GS_PAUSED` are not used: the menu is controlled by a separate `ScreenState` in `main.c` (Section 6).

2.4 Ball

```

49 //Ball's data
50 typedef struct
51 {
52     Vector2 pos;
53     Vector2 vel;
54     float radius;
55     BallState state;
56     int strokes;
57     Vector2 last_safe_pos; //When the ball is in out of bounds state, then it is kept to its
    ↪ final safe position
58 }Ball;

```

Line 52. `pos` – the centre of the ball in world pixels.

Line 53. `vel` – the velocity in pixels per second. Each frame the position moves by `vel * dt`.

Line 54. `radius` – set to 7 pixels in `BallInit`.

Line 55. `state` – one of the `BallState` values above.

Line 56. `strokes` – shots taken on this hole, including penalty strokes.

Line 57. `last_safe_pos` – the last place the ball stopped normally. It is used as a fallback drop position if a hole has no drop zone.

2.5 Wall and Zone

```

61 //Small building block structures
62 typedef struct
63 {
64     Rectangle rect; // width,height,x and y coordinate of the rect's top left corner
    ↪ typedef struct{float x,float y,float width,float height}Rectangle
65     float bounce; // How does the ball react after the collision "restitution"
66 }Wall;
67
68
69 //for zone

```

```

70 typedef struct
71 {
72     Rectangle area; // Built in datatype- area= specifies the region of the current zone
73     SurfaceType type; // User defined datatype - that specifies the type of the surface
                        ↪ whether there is mud or water or green
74     Vector2 wind; // Built in datatype- finds the condition of wind in the current zone
75 }Zone;

```

Line 64. A wall is a solid rectangle.

Line 65. bounce is the *restitution*: how much speed the ball keeps when it hits the wall. 0.72 keeps 72 %, 0.95 is almost a perfect bumper, 0 would stop the ball dead.

Line 72. A zone is a rectangle of ground ...

Line 73. ... with a surface type (sand, ice, ...) ...

Line 74. ... and an optional wind. wind is an *acceleration* in pixels per second². (0, -140) pushes the ball upward a little more every frame while it is inside the zone.

2.6 Hole

```

78 //Entire golf info
79 typedef struct
80 {
81     Wall walls[MAX_WALLS]; //max size of array
82     int wall_count; // actual number of walls
83
84     Zone zones[MAX_ZONES];
85     int zone_count;
86
87     int number; //which number of hole this is
88     int par; //expected number of strokes to finish the hole
89     Vector2 tee_pos; //Where the ball starts
90     Vector2 cup_pos; //position of the hole or target
91     float cup_radius; // When the ball is around the cup(target) , then how much radius will
                        ↪ be considered as a successful shot
92
93     Rectangle bounds; //Whole acceptable region of the rectangular field(boundarries), course
                        ↪ is named as bounds
94     Rectangle drop_zone; //If the ball is in out of bounds state, then move the ball to the
                        ↪ final safest position. This is a special rectangle where the ball is put only when tha
                        ↪ ball goes out of bounds state
95 }Hole;

```

Lines 81–82. A fixed array of 128 walls plus a counter. Only the first wall_count entries are real walls, the rest are unused. This “array + count” pattern is how C stores a list whose length changes, without malloc.

Lines 84–85. The same pattern for zones.

Line 87. number – the hole number shown in the HUD.

Line 88. par – how many strokes a good player needs.

Lines 89–91. Where the ball starts, where the cup is, and the cup’s radius.

Line 93. bounds – the whole playable field. If the ball’s centre leaves it, the ball is out of bounds.

Line 94. drop_zone – after water or out of bounds, the ball is placed at the centre of this rectangle.

2.7 Game

```
98 typedef struct
99 {
100     GameStateId state;
101     Camera2D camera;           //typedef struct{Vector2 target;Vector2 offset;float
                                ↪ rotation;float zoom}Camera2D;
102     Ball ball;
103     Hole hole;
104     int hole_index; //there can be multiple holes per level, so it defines the indices of
                                ↪ each hole
105     int scores[MAX_HOLES]; //we will calculate scores for individual holes, such as scores
                                ↪ [0]=4 that would mean total score for hole 1 is 4.Hence, to get final score we have to
                                ↪ accumulate scores for all holes.
106 }Game;
107
108
109 #endif
```

This struct was planned to hold the whole game, but `main.c` does not use it. The real game state is the `GameApp` struct in `main.c` (Section 6.2). Changing `Game` has no effect.

3 input.h and input.c

The input module is the only place where the mouse is read during play. Every frame it produces one `InputState` that the rest of the game uses.

3.1 input.h

```
1  #ifndef INPUT_H
2  #define INPUT_H
3
4  #include <raylib.h>
5  #include "types.h"
6
7
8
9  typedef struct {
10     Vector2 aim_direction;
11     float aim_angle;
12     Vector2 pointer_world;
13     bool pointer_valid;
14     bool charge_pressed;
15     bool charge_held;
16     bool charge_released;
17
18     bool cancel;
19
20     bool confirm;
21     bool cam_zoom_delta;
22
23 } InputState;
24
25 typedef struct{
26     float aim_angle;
27     float right_held_time;
28     float right_drag_distance;
29     bool right_was_down;
30
31 } InputSystem;
32
```

```

33 void InputInit(InputSystem *sys);
34
35 InputState InputPoll(InputSystem *sys, Vector2 ball_pos, Camera2D cam, float dt);
36
37 Vector2 DirFromAngle(float angle);
38
39
40
41 #endif

```

Lines 1–2, 41. Include guard, as in `types.h`.

Lines 9–23, `InputState`. Everything that happened with the mouse *this frame*:

- `aim_direction`, `aim_angle` – which way the player is aiming, as a unit vector and as an angle in radians;
- `pointer_world` – the mouse position in world coordinates;
- `pointer_valid` – true if the mouse is far enough from the ball to give a clean aim;
- `charge_pressed`, `charge_held`, `charge_released` – the left button went down this frame / is being held / came up this frame;
- `cancel` – a quick right-click happened (cancels a shot);
- `confirm` – a left click, used to skip the “hole done” pause;
- `cam_zoom_delta` – meant for mouse-wheel zoom (see the note in Section 3.3).

Lines 25–31, `InputSystem`. Values that must be *remembered between frames*: the last good aim angle and the timer and distance for the right button. One `InputSystem` lives inside `GameApp` for the whole game, while a new `InputState` is made every frame.

Lines 33–37. The three functions the module offers.

3.2 Constants, `DirFromAngle` and `InputInit`

```

1  #include "input.h"
2  #include <raylib.h>
3  #include <raymath.h>
4  #include <math.h>
5
6
7  #define aim_distance 40.0f
8
9  #define right_click_time 0.25f
10 #define right_click_distance 6.0f
11
12 #define zoom_per_notch 0.1f
13
14
15
16 Vector2 DirFromAngle(float angle){
17     return (Vector2){cosf(angle),sinf(angle)};
18 }
19
20 void InputInit(InputSystem *sys){
21     sys -> aim_angle = 0.0f; /// pore sohoje parsi sir
22     sys -> right_held_time = 0.0f;
23     sys -> right_drag_distance = 0.0f;
24     sys -> right_was_down = false;
25 }

```

Lines 1–4. `raymath.h` gives vector helpers like `Vector2Subtract`; `math.h` gives `cosf`, `sinf`, `atan2f`.

Line 7. If the mouse is closer than 40 pixels to the ball, the aim is not updated (explained below).

Lines 9–10. A right-click counts as “cancel” only if it lasts less than 0.25 seconds and the mouse moves less than 6 pixels.

Line 12. One notch of the mouse wheel would change the zoom by 0.1.

Lines 16–18, `DirFromAngle`. Turns an angle θ into the unit vector $(\cos \theta, \sin \theta)$. Angle 0 points right, $\pi/2$ points down (because screen y grows downward).

Lines 20–25, `InputInit`. Puts every remembered value to zero. The `->` operator accesses a field through a pointer: `sys->aim_angle` is the same as `(*sys).aim_angle`.

3.3 InputPoll

```
27 InputState InputPoll(InputSystem *sys, Vector2 ball_pos, Camera2D cam, float dt){
28     InputState in = {0};
29
30     in.pointer_world = GetScreenToWorld2D(GetMousePosition(), cam);
31
32     // for aiming the ball.
33     Vector2 to_cursor = Vector2Subtract(in.pointer_world, ball_pos);
34     // check if aim is valid even or not
35     if(Vector2LengthSqr(to_cursor) > aim_distance * aim_distance){
36         sys->aim_angle = atan2f(to_cursor.y, to_cursor.x);
37         in.pointer_valid = true;
38     }
39     else{
40         in.pointer_valid = false;
41     }
42     in.aim_angle = sys->aim_angle;
43     in.aim_direction = DirFromAngle(sys->aim_angle);
44
45     in.charge_pressed = IsMouseButtonPressed(MOUSE_LEFT_BUTTON);
46     in.charge_held = IsMouseButtonDown(MOUSE_LEFT_BUTTON);
47     in.charge_released = IsMouseButtonReleased(MOUSE_BUTTON_LEFT);
48
49     in.confirm = IsMouseButtonPressed(MOUSE_BUTTON_LEFT);
50
51     if(IsMouseButtonPressed(MOUSE_BUTTON_RIGHT)){
52         sys->right_held_time = 0.0f;
53         sys->right_drag_distance = 0.0f;
54         sys->right_was_down = true;
55     }
56     if(sys->right_was_down && IsMouseButtonDown(MOUSE_BUTTON_RIGHT)){
57         sys->right_held_time += dt;
58         sys->right_drag_distance = Vector2Length(GetMouseDelta());
59     }
60
61     if(sys->right_was_down && IsMouseButtonReleased(MOUSE_BUTTON_RIGHT)){
62         in.cancel = (sys->right_held_time < right_click_time && sys->right_drag_distance <
        ↪ right_click_distance);
63         sys->right_was_down = false;
64     }
65     in.cam_zoom_delta = GetMouseWheelMove() * zoom_per_notch;
66
67
68     return in;
```

```
69
70 }
```

Line 27. The function receives the remembered state `sys`, the ball's position, the camera and the frame time `dt`, and *returns* a fresh `InputState` by value.

Line 28. `= {0}` sets every field to zero / `false`. So anything not set below is “nothing happened”.

Line 30. `GetMousePosition()` gives the mouse in window pixels. Because the camera can move and zoom, that is not the same as world coordinates. `GetScreenToWorld2D` converts it using the camera.

Line 33. The vector from the ball to the mouse.

Line 35. If that vector is longer than 40 pixels ... Comparing the *squared* length with $40^2 = 1600$ gives the same answer as comparing the length with 40, but avoids a slow square root.

Line 36. ... the aim angle is updated. `atan2f(y, x)` returns the angle of the vector (x, y) in radians, in the range $(-\pi, \pi]$, and it handles every quadrant correctly.

Lines 37–41. Remember whether the pointer gave a valid aim. If the mouse is very close to the ball, tiny movements would swing the angle wildly, so the old angle is kept instead.

Lines 42–43. Copy the (new or old) angle and its direction vector into the result.

Lines 45–47. raylib offers three different questions about a button, and the putter needs all three: `IsMouseButtonPressed` is true only in the one frame the button goes down; `IsMouseButtonDown` is true in every frame while it is held; `IsMouseButtonReleased` is true only in the frame it comes up. (`MOUSE_LEFT_BUTTON` and `MOUSE_BUTTON_LEFT` are two names for the same button.)

Line 49. `confirm` is also a left click, used when a hole is finished.

Lines 51–55. When the right button goes down: reset the timer and distance and remember that it is down.

Lines 56–59. While it stays down: add this frame's time to the timer, and store how far the mouse moved.

Lines 61–64. When it comes up: it was a “cancel” only if it was short *and* the mouse stayed still. This way a right-*drag* (for example to move a camera later) does not cancel a shot.

Line 65. Stores the mouse wheel movement times 0.1.

Line 68. Return the filled `InputState`.

Two small bugs.

- `cam_zoom_delta` is declared `bool` (line 21 of `input.h`) but receives a `float` on line 65. Any non-zero value becomes `true` (1), so the real amount is lost. It should be declared `float`. Nothing uses it yet, so the game is not affected.
- On line 58, `=` stores only the *last frame's* mouse movement. To measure the whole drag it should be `+=`.

4 level.h and level.c

4.1 level.h

```
1  #ifndef LEVEL_H
2  #define LEVEL_H
3
4  #include "input.h"
5  #include "types.h"
6
7  // defining the data structure for COURSE;
8  typedef struct{
9      Hole holes[MAX_HOLES];
10     int hole_count;
11     int current;
12 } Course;
13
14 //editor tool option
15 typedef enum{
16     TOOL_WALL,
17     TOOL_ZONE,
18     TOOL_TEE,
19     TOOL_CUP,
20     TOOL_DROP
21 } Edit_Tool;
22
23 //editor struct
24
25 typedef struct{
26     Edit_Tool tool;
27     SurfaceType zone_type;
28     bool dragging;
29     Vector2 drag_start;
30     int grid_size;
31     int selected;
32 }Editor;
33
34
35 // removed the build hole function... as i will draw map from text files..... so only about
36     ↪ that is declared here...
37
38 bool LoadHoleFromFile(Hole *h,const char *path);
39 bool SaveHoleToFile(const Hole *h, const char *path);
40
41
42 int course_load(Course *c, const char *dir);
43 bool course_advance(Course *c);
44 Hole *course_current(Course *c);
45
46 // for scoring this functions;
47
48
49 const char *score_name(int strokes, int par);
50 int Course_total(const int *scores, int hole_count);
51 int course_to_par(const Course *c, const int *scores);
52 void save_best_scores(const int *best, int hole_count);
53 void load_best_scores(int *best, int hole_count);
54
55
56 //// for surfaces...
57
58 SurfaceType surface_from_name(const char *name);
```

```

59 const char *surface_name(SurfaceType s);
60
61 void EditorInit(Editor *e);
62 void editor_update(Editor *e, Hole *h, const InputState *in);
63
64
65
66
67 #endif

```

Lines 8–12, Course. All holes of the game, how many were loaded, and which one is being played (current starts at 0).

Lines 15–21, Edit_Tool. The five tools of the map editor: draw a wall, draw a zone, place the tee, place the cup, draw the drop zone.

Lines 25–32, Editor. The editor’s state (the name has a typo, “Edtior”, but it is used consistently so it works): current tool, which surface a new zone gets, whether the mouse is dragging and where the drag started, the snapping grid size, and which wall is under the mouse.

Lines 38–62. The public functions. `save_best_scores`, `load_best_scores`, `surface_from_name` and `surface_name` are declared but never written; that is fine as long as nobody calls them.

4.2 The level file format

Each hole is a text file `levels/holeNN.txt`. One command per line; `#` starts a comment.

command	meaning
hole 7	hole number
par 4	par
bounds x y w h	the playable rectangle
tee x y	start position
cup x y r	cup centre and radius
drop x y w h	drop zone
wall x y w h [bounce]	a wall; bounce is optional (default 0.72)
zone t x y w h [wx wy]	a zone of surface number t; wind is optional
end	ignored

Example from `hole07.txt`:

```

tee 160 740
cup 1420 200 16
wall 420 24 40 520 0.8      # a canyon wall
zone 3 460 400 420 240 0 -140 # ice with wind blowing up

```

4.3 AddZone, Validate_hole, AddWall

```

1 #include "level.h"
2 #include "types.h"
3 #include <raymath.h>
4 #include <string.h>
5 #include <raylib.h>
6 #include <stdio.h>
7 #include <math.h>
8

```

```

9
10
11
12
13 // addzone function o lekha baki
14 void AddZone(Hole *h, SurfaceType t, float x, float y, float w, float ht, Vector2 wind){
15     if(h->zone_count >= MAX_ZONES) return;
16     h->zones[h->zone_count++] = (Zone){(Rectangle){x,y,w,ht},t,wind};
17 }
18
19 // for the course loading && file handling for creating the map or course
20
21 bool Validate_hole(Hole *h,const char *path){
22     bool ok = true;
23
24     if(h->par < 1 || h->par > 9) h->par = 3;
25     if(h->cup_radius < 4.0f) h->cup_radius = 14.0f;
26     if(h->bounds.width < 100.0f || h->bounds.height < 100.0f) ok = false;
27     if(!CheckCollisionPointRec(h->tee_pos, h->bounds)) ok = false;
28     if(!CheckCollisionPointRec(h->cup_pos, h->bounds)) ok = false;
29     if(h-> wall_count == 0) ok = false; // im not sure about this one, need to check !!!!!!!
30     return ok;
31 }
32
33 // add_wall function lekha baki ekhono
34 void AddWall(Hole *h, float x, float y, float w, float ht, float bounce){{
35     if(h->wall_count >= MAX_WALLS) return;
36     h->walls[h->wall_count++] = (Wall){(Rectangle){x,y,w,ht},bounce};
37 }
38
39
40
41
42 }

```

Line 14–17, AddZone. If the zone array is full, do nothing. Otherwise write the new zone into position `zone_count` and *then* increase `zone_count` (that is what the post-increment `zone_count++` does). The `(Zone){...}` syntax is a *compound literal*: it builds a whole struct in one expression.

Lines 21–31, Validate_hole. Checks a hole after loading. Two values are *repaired*: a par outside 1–9 becomes 3, and a cup radius under 4 becomes 14. Four problems *reject* the hole: the field is smaller than 100×100 , the tee or the cup is outside the field (`CheckCollisionPointRec` tests whether a point is inside a rectangle), or there are no walls at all. The `path` parameter is not used.

Lines 34–42, AddWall. Same idea as `AddZone`. Note the double brace `{{` on line 34 and the extra `}` on line 42: the body is wrapped in an extra, useless block. It is legal C and changes nothing, but one pair of braces can be removed.

4.4 Parseline

```

44 // taking input from the lines with this function.....
45
46 void Parseline(Hole *h,char *line, int line_no, const char *path){
47     while(*line == ' ') line++;
48     if(*line == '#' || *line == '\0') return;
49
50
51     char k[32];

```

```

52     if(sscanf(line, "%31s",k) != 1) return;
53
54     if(strcmp(k,"hole") == 0){
55         sscanf(line, "%*s %d",&h->number);
56     }
57     else if(strcmp(k, "par") == 0){
58         sscanf(line,"%*s %d",&h->par );
59     }
60     else if(strcmp(k, "bounds") == 0){
61         sscanf(line, "%*s %f %f %f %f",&h->bounds.x,&h->bounds.y,&h->bounds.width, &h->bounds
↪ .height);
62     }
63     else if(strcmp(k, "tee") == 0){
64         sscanf(line, "%*s %f %f",&h->tee_pos.x, &h->tee_pos.y);
65     }
66     else if(strcmp(k, "cup") == 0){
67         sscanf(line, "%*s %f %f %f",&h->cup_pos.x, &h->cup_pos.y, &h->cup_radius);
68     }
69     else if(strcmp(k, "drop") == 0){
70         sscanf(line, "%*s %f %f %f %f", &h->drop_zone.x, &h->drop_zone.y, &h->drop_zone.width
↪ , &h->drop_zone.height);
71     }
72     else if(strcmp(k, "wall") == 0){
73         float x, y, w, ht, b = 0.72f;
74         sscanf(line, "%*s %f %f %f %f %f",&x, &y, &w, &ht, &b);
75         AddWall(h,x,y,w,ht,b);
76     }
77     else if(strcmp(k,"zone") == 0){
78         int zone_number; // here i will take the enum of the zone directly no name conversion
↪ required
79         float x , y ,w , ht, wx = 0.0f, wy = 0.0f;
80         sscanf(line, "%*s %d %f %f %f %f %f",&zone_number,&x,&y,&w, &ht, &wx, &wy);
81         AddZone(h,zone_number,x,y,w,ht,(Vector2){wx, wy});
82     }
83     else if(strcmp(k, "end") == 0){
84         // some decoration here...
85     }
86 }

```

This function reads one line of a level file.

Line 47. Skip leading spaces by moving the line pointer forward.

Line 48. Ignore comment lines and empty lines.

Lines 51–52. Read the first word into k. %31s reads at most 31 characters, so the 32-byte buffer (31 letters + the terminating \0) can never overflow. sscanf returns how many values it read; if not 1, the line is ignored.

Lines 54–85. Compare the word with each command using strcmp (which returns 0 when two strings are equal) and read the numbers that follow.

The %*s trick. Each sscanf reads the *whole line* again. %*s means “read a word and throw it away”, so it skips the command name and the numbers go into the right variables.

Lines 73–75. Optional values: b is set to 0.72 *before* sscanf. If the line has only four numbers, sscanf stops early and b keeps 0.72.

Lines 78–81. The same for zones: wind defaults to (0,0). The zone number is read as an int and passed where a SurfaceType is expected; C converts an int to an enum automatically.

Lines 83–85. end is accepted and does nothing. Any unknown word falls through all the ifs and is silently ignored.

4.5 LoadHoleFromFile

```
88 bool LoadHoleFromFile(Hole *h, const char *path){
89     if(!FileExists(path)) return false;
90
91     char *text = LoadFileText(path);
92     if(!text) return false;
93
94     *h = (Hole){0}; // clearing the struct data, so that no overlap happens in future;
95
96     //setting the default;
97     h->par = 3;
98     h->cup_radius = 14.0f;
99     h->bounds = (Rectangle){0,0,1600,900};
100
101     int line_no = 0;
102     char *line = strtok(text, "\r\n");
103
104     while(line){
105         line_no++;
106         Parseline(h,line,line_no,path);
107         line = strtok(NULL, "\r\n");
108     }
109     UnloadFileText(text);
110
111     return Validate_hole(h,path);
112
113 }
```

Line 89. If the file does not exist, fail.

Line 91. LoadFileText (raylib) reads the whole file into one string in memory.

Line 94. *h = (Hole){0}; clears every field of the hole, so no walls or zones from a previous map are left behind.

Lines 97–99. Default values in case the file does not set them.

Line 102. strtok cuts a string into pieces. The first call gets the text and the separators (\r and \n, so both Windows and Mac/Linux line endings work) and returns the first line.

Lines 104–108. Each later call with NULL continues where the last one stopped and returns the next line, until it returns NULL at the end. Each line goes to Parseline.

Line 109. Free the memory that LoadFileText allocated.

Line 111. Return whether the hole is valid.

4.6 SaveHoleToFile

```
117 #define LEVEL_TEXT_MAX 16384
118 // later
119 bool SaveHoleToFile(const Hole *h, const char *path){
120     char buf[LEVEL_TEXT_MAX];
121     int n = 0;
122     /* ei function pore likhbo, this one for editing option and stuffs for saving */
123     return false;
124 }
```

Not written yet. It always returns false, and buf and n are unused. It is meant to write a hole back to a text file for the editor.

4.7 The course

```
127 int course_load(Course *c, const char *dir){
128     c->hole_count = 0;
129     c->current = 0;
130
131     for(int i = 1; i < MAX_HOLES; i++){
132         const char *path = TextFormat("%s/hole%02d.txt",dir,i);
133         if(!FileExists(path)) break;
134
135         if(LoadHoleFromFile(&c->holes[c->hole_count], path)) c->hole_count++;
136     }
137
138     return c->hole_count;
139 }
140
141 bool course_advance(Course *c){
142     if(c->current + 1 >= c->hole_count) return false;
143     c->current++;
144     return true;
145 }
146
147 Hole *course_current(Course *c){
148     if(c->current < 0 || (c->current >= c->hole_count)) return &c->holes[0];
149     return &c->holes[c->current];
150 }
```

Lines 127–139, `course_load`. Tries `levels/hole01.txt`, `hole02.txt`, ... `TextFormat` works like `sprintf`; `%02d` prints the number with at least two digits and a leading zero (`1 → 01`).

- Line 133: the loop *stops* at the first missing file, so hole numbers must not have gaps.
- Line 135: a file that fails validation is skipped, but loading continues with the next number. Valid holes are stored one after another, so `holes[0]`, `holes[1]`, ... are always filled.
- The loop runs for `i = 1 ... 17`, so at most 17 files are tried.

Lines 141–145, `course_advance`. Go to the next hole. Returns `false` if the current hole was the last one, which tells `main.c` to show the scoreboard.

Lines 147–150, `course_current`. Returns a *pointer* to the current hole, so the caller works on the real hole and not a copy. If `current` is somehow out of range, it safely returns hole 0.

4.8 Scoring

```
153 // now for scoring part
154
155 const char *score_name(int strokes, int par){
156     if(strokes == 1) return "HOLE IN ONE!!!";
157     switch(strokes - par){
158         case - 3 : return "ALBATROSS";
159         case - 2 : return "EAGLE";
160         case - 1 : return "BIRDIE";
161         case 0 : return "PAR";
162         case 1 : return "BOGEY";
163         case 2 : return "DOUBLE BOGEY";
164         case 3 : return "TRIPLE BOGEY";
165         default : return (strokes < par) ? "GREAT" : "You suck,noob";
166     }
```

```

167     }
168 }
169
170
171 int Course_total(const int *scores, int hole_count){
172     int total = 0;
173     for(int i = 0; i < hole_count; i++){
174         if(scores[i] > 0){
175             total += scores[i];
176         }
177     }
178     return total;
179 }
180
181 // this function needs some reading for mee...
182 int course_to_par(const Course *c, const int *scores){
183     int diff = 0;
184     for(int i = 0; i < c->hole_count; i++){
185         if(scores[i] > 0) diff+= scores[i] - c->holes[i].par;
186     }
187
188     return diff;
189 }

```

Lines 155–168, `score_name`. One stroke is always “HOLE IN ONE!!!”. Otherwise `strokes - par` picks the name: `-3` albatross, `-2` eagle, `-1` birdie, `0` par, `+1` bogey, `+2` double bogey, `+3` triple bogey. The `default` case handles everything else: better than `-3` gives “GREAT”, worse than `+3` gives the insult. (case `- 3` with a space is simply the number `-3`.)

Lines 171–179, `Course_total`. Adds up all scores. A score of `0` means “not played yet” and is skipped.

Lines 182–189, `course_to_par`. Adds up `score - par` for played holes. A result of `-2` means two under par for the round so far.

4.9 The editor

```

195 // editor
196
197 void EditorInit(Editor *e){
198     e->tool = TOOL_WALL;
199     e->zone_type = SURF_SAND;
200     e->dragging = false;
201     e->grid_size = 10;
202     e->selected = -1;
203 }
204
205 Vector2 Snap2Grid(Vector2 p, int grid){
206     if(grid <= 1) return p;
207     return (Vector2){ roundf(p.x / grid) * grid, roundf(p.y / grid) * grid};
208 }
209
210 Rectangle RectFromCorners(Vector2 a, Vector2 b){
211     return (Rectangle){
212         fminf(a.x,b.x), fminf(a.y,b.y),fabsf(b.x - a.x),fabsf(b.y - a.y)
213     };
214 }
215
216 // its simply like removing a element from middle of an array and shifting the position
217 void RemoveWall(Hole *h, int index){

```

```

218     if(index < 0 || index >= h->wall_count) return;
219     for(int i = index; i < h->wall_count - 1; i++){
220         h->walls[i] = h->walls[i + 1];
221     }
222     h->wall_count--;
223 }
224
225 int WallAtPoint(const Hole *h, Vector2 p){
226     for(int i = h->wall_count - 1; i >= 0; i--){
227         if(CheckCollisionPointRec(p, h->walls[i].rect)) return i;
228     }
229     return -1;
230 }
231
232 // for undoing things while the editor is open ... hell yeah , we are making our own map
233 Hole UndoBuffer;
234 bool UndoValid = false;
235
236 void PushUndo(const Hole *h){
237     UndoBuffer = *h;
238     UndoValid = true;
239 }
240
241 void PopUndo(Hole *h){
242     if(!UndoValid) return;
243     Hole temporary = *h;
244     *h = UndoBuffer;
245     UndoBuffer = temporary;
246 }

```

Lines 197–203, EditorInit. Start with the wall tool, sand as the zone surface, a 10-pixel grid and nothing selected (−1 means “no wall”).

Lines 205–208, Snap2Grid. Rounds a point to the nearest grid line. With a grid of 10, $x = 347$ becomes $\text{round}(34.7) \cdot 10 = 350$. A grid of 1 means no snapping.

Lines 210–214, RectFromCorners. The user may drag in any direction. `fminf` takes the smaller coordinates as the top-left corner and `fabsf` (absolute value) gives a positive width and height.

Lines 217–223, RemoveWall. Deletes wall number `index` by shifting every later wall one place to the left, then reducing the count. Not called anywhere yet.

Lines 225–230, WallAtPoint. Finds the wall under a point. It searches from the *last* wall backwards, so if walls overlap, the one drawn on top is found first. Returns −1 if there is none.

Lines 233–234. A global copy of one hole, used for a one-step undo.

Lines 236–239, PushUndo. Saves a copy of the hole before a change.

Lines 241–246, PopUndo. Swaps the current hole with the saved copy. Because it swaps (using `temporary`), calling it twice is a redo. Not called anywhere yet.

```

248 void editor_update(Editor *e, Hole *h, const InputState *in){
249     Vector2 p = Snap2Grid(in->pointer_world, e->grid_size);
250     e->selected = WallAtPoint(h, in->pointer_world);
251
252
253     // for the tools
254     if(IsKeyPressed(KEY_ONE)) e->tool = TOOL_WALL;

```

```

255     if(IsKeyPressed(KEY_TWO)) e->tool = TOOL_ZONE;
256     if(IsKeyPressed(KEY_THREE)) e->tool = TOOL_TEE;
257     if(IsKeyPressed(KEY_FOUR)) e->tool = TOOL_CUP;
258     if(IsKeyPressed((KEY_FIVE))) e->tool = TOOL_DROP;
259
260     // cycle of zone_type baby
261
262     if(IsKeyPressed(KEY_TAB)){
263         e->zone_type = (SurfaceType)((e->zone_type + 1) % SURF_COUNT);
264     }
265
266     //// another cycle but with if else loop _ for grid sizing
267     if(IsKeyPressed(KEY_G)){
268         if(e->grid_size == 1) e->grid_size = 5;
269         else if(e->grid_size == 5) e->grid_size = 10;
270         else if(e->grid_size == 10) e->grid_size = 20;
271         else e->grid_size = 1;
272     }
273
274     // main edit starts here...
275     if(IsMouseButtonPressed(MOUSE_BUTTON_LEFT)){
276         PushUndo(h);
277         e->dragging = true;
278         e->drag_start = p;
279     }
280
281     if(e->dragging && IsMouseButtonReleased(MOUSE_BUTTON_LEFT)){
282         e->dragging = false;
283         Rectangle r = RectFromCorners(e->drag_start, p);
284
285         switch(e->tool){
286             case TOOL_TEE:
287                 h->tee_pos = p;
288                 break;
289             case TOOL_CUP:
290                 h->cup_pos = p;
291                 break;
292             case TOOL_DROP:
293                 if(r.width >= 4.0f && r.height >= 4.0f){
294                     h->drop_zone = r;
295                 }
296                 break;
297             case TOOL_WALL :
298                 if(r.width >= 4.0f && r.height >= 4.0f)
299                     AddWall(h,r.x,r.y,r.width,r.height,0.72f);
300                 break;
301             case TOOL_ZONE:
302                 if(r.width >= 4.0f && r.height >= 4.0f){
303                     AddZone(h,e->zone_type, r.x, r.y, r.width, r.height,Vector2Zero());
304                 }
305                 break;
306         }
307     }
308 }
309
310 // forwhat
311
312
313 }

```

Line 249. The mouse position snapped to the grid.

Line 250. Remember which wall is under the mouse (for highlighting).

Lines 254–258. Keys 1–5 choose the tool.

Lines 262–264. Tab cycles the zone surface: $0 \rightarrow 1 \rightarrow \dots \rightarrow 6 \rightarrow 0$. The % (remainder) operator makes it wrap around, and SURF_COUNT is why it wraps at the right number.

Lines 267–272. G cycles the grid: $1 \rightarrow 5 \rightarrow 10 \rightarrow 20 \rightarrow 1$.

Lines 275–279. When the left button goes down: save an undo copy and remember where the drag started.

Lines 281–283. When it comes up: build the rectangle between the start and the current point.

Lines 285–307. Use it according to the tool. Tee and cup use only the release point. Walls, zones and the drop zone need a rectangle at least 4×4 pixels, so a plain click does not create an invisible object.

Status. main.c calls EditorInit but never calls editor_update, so the editor is not active in the game yet, even though the HUD mentions “F1 editor”.

5 putter.h and putter_simple.c

5.1 putter.h

```
1  #ifndef PUTTER_H
2  #define PUTTER_H
3
4
5  #include <raylib.h>
6  #include <raymath.h>
7  #include <math.h>
8  #include <stdio.h>
9
10
11 #include "ball.h"
12 #include "types.h"
13 #include "input.h"
14
15
16 typedef enum{
17     putter_idle,
18     putter_backswing,
19     putter_strike,
20     putter_follow,
21     putter_recover
22 } PutterPhase;
23
24 typedef struct{
25     PutterPhase phase;
26     float t; // seconds inside the current phase
27     float power;
28     float launch_power;
29     float aim_angle;
30     float offset;
31     float dir; // +1 while the bar is filling, -1 while it is emptying
32     bool fired;
33 } Putter;
34
35 void putter_init(Putter *p);
36 void putter_update(Putter *p, const InputState *in, Ball *ball, float dt);
37 bool PutterIsCharging(const Putter *p);
```

```

38 float PutterPower(const Putter *p);
39 void putter_draw(const Putter *p,const Ball *ball);
40
41 #endif

```

Lines 16–22, PutterPhase. The phases of a swing. The simple putter uses only `putter_idle` (waiting) and `putter_backswing` (charging power). `strike`, `follow` and `recover` were for the animated club in `unused/putter_anim.c`.

Lines 24–33, Putter.

- `phase` – the current phase;
- `t`, `offset` – only used by the animated version;
- `power` – the power bar, from 0 to 1;
- `launch_power` – the power actually used for the last shot;
- `aim_angle` – a copy of the input aim;
- `dir` – +1 while the bar fills, –1 while it empties;
- `fired` – true in the frame a shot was taken.

Lines 35–39. The functions. Parameters marked `const` are only read, never changed.

5.2 putter_simple.c

```

1  #include "putter.h"
2  #include "ball.h"
3
4
5
6
7  #define CHARGE_TIME 1.15f
8  #define MIN_POWER 0.05f
9
10
11 void putter_init(Putter *p){
12     p->phase = putter_idle;
13     p->t = p->power = p->launch_power = p->aim_angle = p->offset = 0.0f;
14     p->dir = 1.0f;
15     p->fired = false;
16 }
17
18 bool PutterIsCharging(const Putter *p){
19     return p->phase == putter_backswing;
20 }
21
22 float PutterPower(const Putter *p){
23     return p->power;
24 }

```

Line 7. Filling the bar from 0 to 1 takes 1.15 seconds.

Line 8. The weakest possible shot is 5% power, so a very quick click still moves the ball.

Lines 11–16, putter_init. Start idle with everything at zero. Line 13 is a chained assignment: it is read from right to left, so all five fields become 0.

Lines 18–20. The putter is “charging” exactly when it is in the backswing. The HUD calls this to decide whether to draw the power bar.

Lines 22–24. A simple getter for the current power.

```

26 void putter_update(Putter *p, const InputState *in, Ball *ball, float dt){
27     p->aim_angle = in->aim_angle;
28
29     bool addressable = ball->state == BALL_AIM;
30
31     if(p->phase == putter_idle){
32         p->power = 0.0f;
33         p->launch_power = 0.0f;
34         p->fired = false;
35
36         if(addressable && in->charge_pressed){
37             p->phase = putter_backswing;
38             p->dir = 1.0f;
39         }
40         return;
41     }
42
43     // charging logic ekhanee...
44     if(in->charge_held){
45         p->power += p->dir * dt / CHARGE_TIME;
46         if(p->power > 1.0f){ p->power = 1.0f; p->dir = -1.0f; }
47         if(p->power < 0.0f){ p->power = 0.0f; p->dir = 1.0f; }
48     }
49
50     if(in->cancel){
51         p->power = 0.0f;
52         p->phase = putter_idle;
53         return;
54     }
55
56     if(in->charge_released){
57         p->launch_power = (p->power < MIN_POWER) ? MIN_POWER : p->power;
58         p->fired = true;
59         BallLaunch(ball, DirFromAngle(p->aim_angle), p->launch_power);
60         p->power = 0.0f;
61         p->phase = putter_idle;
62     }
63 }
64
65 void putter_draw(const Putter *p, const Ball *ball){
66     // nothing to draw for now, no club, no animation
67     (void)p;
68     (void)ball;
69 }

```

Line 27. Always follow the player's aim.

Line 29. The ball can only be hit when it is resting.

Lines 31–41, idle phase. Keep the power at zero. If the ball is resting and the left button was *just pressed*, switch to the backswing with the bar filling. **return** ends the function here, so the charging code below does not run in the same frame.

Lines 44–48, charging. While the button is held, change the power by $\text{dir} \cdot dt / 1.15$. At 60 FPS, $dt \approx 0.0167$ s, so the power grows by about 0.0145 per frame. When it passes 1 it is clamped to 1 and **dir** flips to -1 ; when it drops below 0 it flips back. So the bar goes up and down like a ping-pong ball until the player lets go.

Lines 50–54, cancel. A quick right-click throws the charge away and returns to idle.

Lines 56–62, firing. When the left button is released:

- line 57: use the current power, but at least MIN_POWER (the `? : c` means “if a then b else c”);
- line 59: BallLaunch in `ball.c` sets the ball’s velocity to

$$\vec{v} = (\cos \theta, \sin \theta) \cdot \text{power} \cdot 1300 \text{ px/s},$$

switches the ball to BALL_ROLL and adds one stroke;

- lines 60–61: reset the bar and go back to idle.

Lines 65–69, putter_draw. Draws nothing (there is no club picture). `(void)p;` tells the compiler “I know this parameter is unused”, which removes a warning.

Example. Holding for 0.575 s gives power $0.575/1.15 = 0.5$, so the ball starts at 650 px/s. Holding for 1.6 s: the bar reached 1 at 1.15 s and has been falling for 0.45 s, so power = $1 - 0.45/1.15 \approx 0.61$.

How far does it roll? With constant friction a , a ball starting at speed v stops after $d = v^2/(2a)$. A full shot on the fairway ($a = 300$) rolls $1300^2/600 \approx 2800$ px; on mud ($a = 2600$) only about 325 px.

6 The menu in `main.c`

6.1 ScreenState

```

1  #include "raylib.h"
2  #include "raymath.h"
3
4
5  #include "types.h"
6  #include "ball.h"
7  #include "level.h"
8  #include "input.h"
9  #include "putter.h"
10 #include "render.h"
11
12
13 #define STROKE_LIMIT_OVER_PAR 5
14 #define HOLE_DONE_PAUSE 1.60f
15
16
17 typedef enum {
18     SCREEN_MAIN_MENU,
19     SCREEN_GAME,
20     SCREEN_TUTORIAL,
21     SCREEN_ABOUT
22 } ScreenState;
```

Lines 1–10. `main.c` includes every module because it connects them all.

Line 13. A hole ends automatically after `par + 5` strokes.

Line 14. The “BIRDIE!” message stays for 1.6 seconds.

Lines 17–22. Which screen is on: the main menu, the game, the tutorial picture or the about picture. This is the top-level state; `GameStateId` (from `types.h`) only matters when the screen is `SCREEN_GAME`.

6.2 GameApp

```
25 typedef struct {
26     GameStateId state;
27     Course course;
28     Ball ball;
29     Putter putter;
30     InputSystem input_sys;
31     Editor editor;
32     RenderState render;
33     int scores[MAX_HOLES];
34     int best[MAX_HOLES];
35     float hole_done_t;
36     float prev_speed;
37
38     //Menu variables
39     ScreenState screen;
40     Texture2D menuBackground;
41     Texture2D tutorialImage;
42     Texture2D aboutImage;
43     Texture2D windSprite;
44     Music menuMusic;
45     Music gameMusic;
46     bool muted;
47 } GameApp;
```

Lines 26–36. Game data: round state, the course, the ball, the putter, the input memory, the editor, the camera, the score of each hole, the best score of each hole, the timer for the “hole done” pause, and `prev_speed` (not used yet).

Lines 39–46. Menu data: the current screen, three pictures for the menu, tutorial and about screens, the wind animation, the two music tracks, and whether sound is muted.

6.3 DrawFullscreenTexture

```
50 void DrawFullscreenTexture(Texture2D texture)
51 {
52     Rectangle source = {0,0,(float)texture.width,(float)texture.height};
53     Rectangle destination = {0,0,(float)GetScreenWidth(),(float)GetScreenHeight()};
54     Vector2 origin = {0,0};
55     DrawTexturePro(texture,source,destination,origin,0.0f,WHITE);
56 }
```

`DrawTexturePro` copies the `source` rectangle of the picture (here the whole picture) into the `destination` rectangle on screen (here the whole window). If the sizes differ, the image is stretched, so the background always fills the window, even after resizing. `origin` is the rotation point and `0.0f` is the rotation angle. `WHITE` as the tint means “draw the original colours”.

6.4 DrawMenuButton

```
59 bool DrawMenuButton(Rectangle rect,const char *text)
60 {
61     Vector2 mouse = GetMousePosition();
62     bool hovered = CheckCollisionPointRec(mouse,rect);
63
64     Color buttonColor;
65     if (hovered) buttonColor = (Color){70,120,70,230};
66     else buttonColor = (Color){30,50,30,210};
67 }
```

```

68     DrawRectangleRounded(rect,0.20f,10,buttonColor);
69     DrawRectangleRoundedLines(rect,0.20f,10,(Color){220,220,180,255});
70
71     int fontSize = 30;
72     int textWidth = MeasureText(text,fontSize);
73     DrawText(text,(int)(rect.x + rect.width/2 - textWidth/2),(int)(rect.y + rect.height/2 -
    ↪  fontSize/2),fontSize,WHITE);
74     if (hovered && IsMouseButtonPressed(MOUSE_BUTTON_LEFT)) return true;
75     return false;
76 }

```

Lines 61–62. Is the mouse over the button? (Menu buttons use screen coordinates, so no camera conversion is needed.)

Lines 64–66. A lighter green when hovered, a darker green otherwise. The fourth number in a `Color` is alpha (opacity, 0–255).

Lines 68–69. Draw the rounded button and its light outline. `0.20f` is the roundness and `10` the number of segments used to draw each corner.

Lines 71–73. Centre the text: `MeasureText` gives the text width in pixels, so the text starts at centre – width/2. Vertically it starts half a font size above the centre.

Lines 74–75. Return `true` only in the frame the button is clicked.

This style is called an *immediate-mode* GUI: there are no button objects stored anywhere. Each frame the function is called again, draws the button and answers “was I clicked right?”. To add a button you just add one more call.

6.5 StartHole

```

79 void StartHole(GameApp *g)
80 {
81     BallInit(&g->ball,course_current(&g->course)->tee_pos);
82     putter_init(&g->putter);
83     g->render.hole_time = 0.0f;
84     g->render.cam.target = g->ball.pos;
85     g->state = GS_PLAYING;
86 }

```

Used when a new hole begins: put the ball on the tee (`BallInit` also resets the stroke count to 0), reset the putter, restart the hole timer, jump the camera to the ball, and go back to `GS_PLAYING`.

6.6 Music functions

```

89 void StartGameMusic(GameApp *g)
90 {
91     StopMusicStream(g->menuMusic);
92     if (!g->muted) PlayMusicStream(g->gameMusic);
93 }
94
95
96 void StartMenuMusic(GameApp *g)
97 {
98     StopMusicStream(g->gameMusic);
99     if (!g->muted) PlayMusicStream(g->menuMusic);
100 }
101

```

```

102
103 void ToggleMute(GameApp *g)
104 {
105     g->muted = !g->muted;
106
107     if (g->muted) {
108         SetMusicVolume(g->menuMusic,0.0f);
109         SetMusicVolume(g->gameMusic,0.0f);
110     }
111     else {
112         SetMusicVolume(g->menuMusic,0.45f);
113         SetMusicVolume(g->gameMusic,0.35f);
114     }
115 }

```

Lines 89–93, StartGameMusic. Stop the menu song and play the game song, unless muted.

Lines 96–100, StartMenuMusic. The opposite.

Lines 103–115, ToggleMute. Flip muted with `!`, then set both volumes to 0 or back to normal. The music keeps playing silently, so un-muting continues the song instead of restarting it.

6.7 GameInit

```

118 void GameInit(GameApp *g)
119 {
120     *g = (GameApp){0};
121
122     course_load(&g->course,"levels");
123
124     for (int i = 0;i < MAX_HOLES;i++) g->scores[i] = 0;
125
126     InputInit(&g->input_sys);
127     EditorInit(&g->editor);
128     BallInit(&g->ball,course_current(&g->course)->tee_pos);
129     putter_init(&g->putter);
130     RenderInit(&g->render,g->ball.pos);
131     g->state = GS_PLAYING;
132     g->screen = SCREEN_MAIN_MENU;
133     g->muted = false;
134
135     g->menuBackground = LoadTexture("menu.png");
136     g->tutorialImage = LoadTexture("tutorials.png");
137     g->aboutImage = LoadTexture("GolfGameCredit.png");
138     g->windSprite = LoadTexture("wind_sprite.png");
139     SetTextureFilter(g->windSprite,TEXTURE_FILTER_BILINEAR);
140     g->menuMusic = LoadMusicStream("golfmusic.mp3");
141     g->gameMusic = LoadMusicStream("gamemusic.mp3");
142     g->menuMusic.looping = true;
143     g->gameMusic.looping = true;
144     SetMusicVolume(g->menuMusic,0.45f);
145     SetMusicVolume(g->gameMusic,0.35f);
146     PlayMusicStream(g->menuMusic);
147 }

```

Line 120. Clear the whole struct.

Line 122. Load all holes from the `levels` folder.

Lines 126–131. Initialise each module and set the round state to playing.

Line 132. The game starts on the **main menu**, not in play.

Lines 135–141. Load the pictures, the wind animation and the two songs. Line 139 makes the wind sprite use smooth (bilinear) scaling. The paths are relative to the folder the program is *started from*, so run it from the project folder (`make run` or `./golf`). A file must also really be the format its name says: a JPEG renamed to `.png` will not load.

Lines 142–145. Both songs loop, with the menu slightly louder.

Line 146. Start the menu music.

6.8 DrawMainMenu

```
150 bool DrawMainMenu(GameApp *g)
151 {
152     DrawFullscreenTexture(g->menuBackground);
153     DrawRectangle(0,0,GetScreenWidth(),GetScreenHeight(),(Color){0,0,0,70});
154     const char *title = "MINI GOLF";
155     int titleFontSize = 70;
156     int titleWidth = MeasureText(title,titleFontSize);
157     DrawText(title,GetScreenWidth()/2 - titleWidth/2,80,titleFontSize,WHITE);
158     float buttonWidth = 260.0f;
159     float buttonHeight = 60.0f;
160     float centerX = GetScreenWidth()/2.0f - buttonWidth/2.0f;
161     Rectangle playButton = {centerX,220,buttonWidth,buttonHeight};
162     Rectangle tutorialButton = {centerX,300,buttonWidth,buttonHeight};
163     Rectangle aboutButton = {centerX,380,buttonWidth,buttonHeight};
164     Rectangle soundButton = {centerX,460,buttonWidth,buttonHeight};
165     Rectangle exitButton = {centerX,540,buttonWidth,buttonHeight};
166     if (DrawMenuButton(playButton,"PLAY")) {
167         g->screen = SCREEN_GAME;
168         StartGameMusic(g);
169     }
170     if (DrawMenuButton(tutorialButton,"TUTORIAL")) g->screen = SCREEN_TUTORIAL;
171     if (DrawMenuButton(aboutButton,"ABOUT US")) g->screen = SCREEN_ABOUT;
172     const char *soundText;
173     if (g->muted) soundText = "SOUND: OFF";
174     else soundText = "SOUND: ON";
175     if (DrawMenuButton(soundButton,soundText)) ToggleMute(g);
176     if (DrawMenuButton(exitButton,"EXIT")) return true;
177     return false;
178 }
```

Line 152. The background picture fills the window.

Line 153. A black rectangle with alpha 70 (about 27 % opaque) over the whole window darkens the picture so the white text is easier to read.

Lines 154–157. The title “MINI GOLF” in size 70, centred horizontally, 80 pixels from the top.

Lines 158–165. Five buttons, 260×60 pixels, all centred horizontally, with their tops at $y = 220, 300, 380, 460, 540$ (80 pixels apart). `centerX` is recomputed every frame, so the buttons stay centred when the window is resized.

Lines 166–169, PLAY. Switch to the game screen and the game music.

Lines 170–171. TUTORIAL and ABOUT US switch screens.

Lines 172–175, SOUND. The label is chosen from `muted` each frame, so it always shows the current setting. Clicking it calls `ToggleMute`.

Line 176, EXIT. Returns `true`; `main` uses this to end the loop.

Line 177. Otherwise return `false`: keep running.

6.9 Tutorial and About screens

```
181 void DrawTutorialScreen(GameApp *g)
182 {
183     DrawFullscreenTexture(g->tutorialImage);
184     Rectangle backButton = {30,30,160,55};
185     if (DrawMenuButton(backButton,"BACK")) g->screen = SCREEN_MAIN_MENU;
186 }
187
188
189 void DrawAboutScreen(GameApp *g)
190 {
191     DrawFullscreenTexture(g->aboutImage);
192     Rectangle backButton = {30,30,160,55};
193     if (DrawMenuButton(backButton,"BACK")) g->screen = SCREEN_MAIN_MENU;
194 }
```

Each screen shows one full-window picture and a BACK button in the top-left corner (at $x = 30$, $y = 30$, 160×55 pixels). Clicking BACK returns to the main menu.

6.10 The main loop: choosing the screen

```
197 int main(void)
198 {
199     SetConfigFlags(FLAG_MSAA_4X_HINT | FLAG_WINDOW_RESIZABLE);
200
201
202     InitWindow(1280,720,"Mini Golf By Akif Rafin");
203     InitAudioDevice();
204     SetExitKey(KEY_NULL);
205     SetTargetFPS(60);
206
207     GameApp game;
208     GameInit(&game);
209     bool shouldQuit = false;
210
211
212     while (!WindowShouldClose() && !shouldQuit) {
213         float dt = GetFrameTime();
214         if (dt > 0.05f) dt = 0.05f;
215         UpdateMusicStream(game.menuMusic);
216         UpdateMusicStream(game.gameMusic);
217         if (game.screen == SCREEN_MAIN_MENU) {
218             BeginDrawing();
219             ClearBackground(BLACK);
220             shouldQuit = DrawMainMenu(&game);
221             EndDrawing();
222             continue;
223         }
224         if (game.screen == SCREEN_TUTORIAL) {
225             if (IsKeyPressed(KEY_ESCAPE)) game.screen = SCREEN_MAIN_MENU;
226             BeginDrawing();
227             ClearBackground(BLACK);
228             DrawTutorialScreen(&game);
229             EndDrawing();
230             continue;
231         }
232         if (game.screen == SCREEN_ABOUT) {
233             if (IsKeyPressed(KEY_ESCAPE)) game.screen = SCREEN_MAIN_MENU;
234             BeginDrawing();
235             ClearBackground(BLACK);
236             DrawAboutScreen(&game);
```

```

237     EndDrawing();
238     continue;
239 }
240 Hole *hole = course_current(&game.course);
241 InputState in = InputPoll(&game.input_sys, game.ball.pos, game.render.cam, dt);
242 if (IsKeyPressed(KEY_ESCAPE)) {
243     game.screen = SCREEN_MAIN_MENU;
244     StartMenuMusic(&game);
245     continue;
246 }

```

Line 199. Ask for 4× anti-aliasing (smoother edges) and a resizable window.

Lines 202–203. Open the window and the audio device. Audio must be started before any music is loaded.

Line 204. By default raylib closes the window when **Esc** is pressed. **SetExitKey(KEY_NULL)** turns that off, so **Esc** can be used for “back to menu”.

Line 205. Run at 60 frames per second.

Lines 207–209. Create and initialise the game. **shouldQuit** becomes **true** when **EXIT** is clicked.

Line 212. Keep looping until the window’s close button is pressed or **EXIT** is clicked.

Lines 213–214. Frame time, clamped to 0.05s so that one slow frame cannot move the ball too far.

Lines 215–216. raylib streams music in small pieces. **UpdateMusicStream** must be called every frame for each song, otherwise the music stutters and stops.

Lines 217–223, main menu. Draw the menu. **continue** jumps straight to the next loop iteration, so none of the game code below runs while the menu is open. The game is frozen, and **PLAY** resumes it exactly where it was.

Lines 224–239. The tutorial and about screens work the same way; **Esc** also goes back to the menu.

Lines 240–246. From here on the screen is **SCREEN_GAME**. The input is read, and **Esc** returns to the menu and switches back to the menu music.

The screens and how you move between them:

from	action	to
main menu	PLAY	game
main menu	TUTORIAL	tutorial
main menu	ABOUT US	about
main menu	EXIT	program ends
tutorial / about	BACK or Esc	main menu
game	Esc	main menu

6.11 Cleaning up

```

309     StopMusicStream(game.menuMusic);
310     StopMusicStream(game.gameMusic);
311     UnloadMusicStream(game.menuMusic);
312     UnloadMusicStream(game.gameMusic);

```

```
$13     UnloadTexture(game.menuBackground);
$14     UnloadTexture(game.tutorialImage);
$15     UnloadTexture(game.aboutImage);
$16     UnloadTexture(game.windSprite);
$17
$18     CloseAudioDevice();
$19     CloseWindow();
$20
$21     return 0;
$22 }
```

After the loop ends, the music is stopped, every song and picture is unloaded from memory, and the audio device and window are closed. Every `Load...` call has a matching `Unload...` call.